



Politechnika Wrocławska

Sterowanie procesami dyskretnymi Automatyka i Robotyka, W12N	Zadanie nr 2 Permutacyjny problem przepływowy uszeregowania zadań	
Jakub Borsukiewicz 278319 Jan Farbotko 278325	Termin zajęć: Wt. Tn. 11:15	Prowadzący: dr inż. Agnieszka Wielgus

Spis treści

1	Opis problemu matematycznego	3
2	Zaimplementowane algorytmy	3
2.1	Przegląd zupełny	3
2.2	Algorytm NEH	3
2.3	Algorytm Johnsona dla $m=2$	4
2.4	Algorytm FNEH (NEH z akceleracją)	4
2.5	Algorytm podstawowy podziału i ograniczeń	5
2.6	Algorytm meta-heurystyczny - Akceptacja Progu	5
2.7	Algorytm meta-heurystyczny - VNS (Variable Neighborhood Search)	6
3	Testy eksperymentalne	6
3.1	Wyniki testów działania algorytmów dla $m>2$	6
3.2	Wyniki testów dla algorytmu Johnsona dla $m=2$	8
4	Wnioski	9

1 Opis problemu matematycznego

Dany jest zbiór n zadań $J = 1, \dots, n$ i m procesorów $M = M_1, \dots, M_m$, gdzie każde zadanie j składa się ze zbioru m operacji oraz każda z tych operacji musi być wykonana na każdym z procesorów. Zakłada się, że każda operacja może rozpocząć się tylko wtedy gdy poprzednia operacja się zakończyła.

W podanym zadaniu podjęto się problemu permutacyjnego, czyli każdy procesor musi wykonać operacje w tej samej kolejności.

Zakłada się również, że każdy procesor może jednocześnie wykonywać maksymalnie jedną operację.

Czas wykonania zadania j - p_j opisany jest liczbą rzeczywistą.

Permutacja π to uszeregowanie zadań na procesorach dla m -procesorowego permutacyjnego pomiaru przepływowego. Dla każdego $\pi(i)$, czyli uszeregowanego na pozycji i w permutacji π można określić czas zakończenia jego wykonywania $C^{(z)}$ na procesorze M_z , gdzie $z = 1, \dots, m$.

Celem problemu jest znalezienie takiego uszeregowania zadań π , które minimalizuje kryterium długości uszeregowania.

2 Zaimplementowane algorytmy

Zaimplementowano różne algorytmy, których celem była minimalizacja kryterium długości uszeregowania zadań π na procesorach.

2.1 Przegląd zupełny

- **Zasada działania:** Algorytm generuje wszystkie możliwe permutacje zadań i dla każdej z nich wyznacza wartość kryterium C_{max} .
- **Cechy szczególne:** Gwarantowane jest optymalne rozwiązanie poprzez pełne przeszukiwanie przestrzeni rozwiązań.
- **Złożoność obliczeniowa:** $O(n! \cdot m)$ - wykładniczy wzrost czasu obliczeń sprawia, że algorytm jest nieprzydatny dla instancji o większej liczbie zadań.

2.2 Algorytm NEH

- **Zasada działania:** Na początku zadania są sortowane malejąco według sumy ich czasów wykonania na wszystkich procesorach. Następnie tworzone są uporządkowane listy, gdzie kolejne zadania są wstawiane

na wszystkie możliwe pozycje. Wybierana jest pozycja która minimalizuje wartość C_{max} .

- **Cechy szczególne:** Jedna z najlepszych dotychczas znanych heurystyk dla problemu przepływowego, która daje wyniki bliski optymalnemu w krótkim czasie.
- **Złożoność obliczeniowa:** $O(n^3 \cdot m)$ w klasycznej implementacji z pełnym przeliczaniem kryterium przy każdym wstawieniu.

2.3 Algorytm Johnsona dla $m=2$

- **Zasada działania:** Algorytm dzieli zadania na dwa zbiory: J_L (zadania, dla których czas na pierwszym procesorze jest mniejszy niż na drugim) oraz J_R (pozostałe zadania). Zbiór J_L jest sortowany rosnąco po czasie pierwszej operacji, a zbiór J_R malejąco po czasie drugiej operacji. Ostateczna permutacja powstaje z połączenia obu zbiorów (J_L na początku, J_R na końcu).
- **Cechy szczególne:** Konstruuje uszeregowanie optymalne dla problemu dwuprocesorowego ($m = 2$).
- **Złożoność obliczeniowa:** $O(n \log n)$, wynikająca z konieczności posortowania podzbiorów zadań.

2.4 Algorytm FNEH (NEH z akceleracją)

- **Zasada działania:** Na początku tak jak w algorytmie NEH zadania są sortowane malejąco po sumie czasów. Następnie w celu optymalizacji tworzone są dwie tablice: przejścia w przód oraz przejścia w tył. Na koniec wyznacza się wartość kryterium po wstawieniu zadania na dowolną pozycję.
- **Cechy szczególne:** Pozwala na znaczne przyspieszenie obliczeń fazy konstrukcyjnej bez jakiegokolwiek straty na jakości generowanego rozwiązania (wynik taki sam jak w klasycznym NEH).
- **Złożoność obliczeniowa:** Zredukowana do $O(n^2 \cdot m)$ dzięki akceleracji wyznaczania wartości kryterium.

2.5 Algorytm podstawowy podziału i ograniczeń

- **Dolne ograniczenie (LB):** Dla każdego węzła drzewa obliczane jest strukturalne dolne ograniczenie osobno dla każdej maszyny k . Składa się ono z: czasu zakończenia dotychczas uszeregowanych zadań na maszynie k , sumy czasów pozostałych (nieuszeregowanych) zadań na maszynie k oraz minimalnego "ogona" (czasu, jaki pozostałe zadania muszą jeszcze spędzić na maszynach od $k+1$ do m). Ostateczne LB węzła to wartość maksymalna z ograniczeń wyznaczonych dla wszystkich maszyn.
- **Górne ograniczenie (UB):** Jako początkowe górne ograniczenie (najlepszy znany dotychczas pełny wynik) przyjmowany jest wynik działania algorytmu NEH. Jest ono aktualizowane w trakcie przeszukiwania, gdy algorytm dotrze do liścia drzewa o LB mniejszym niż aktualne UB .
- **Sposób podziału:** Podział (generowanie dzieci) polega na wyborze kolejnego zadania ze zbioru zadań pozostałych i doklejeniu go na koniec dotychczasowej sekwencji cząstkowej.
- **Przeglądanie drzewa:** Wykorzystano strategię przeszukiwania najlepiej dopasowanego (*Best-First Search*), zaimplementowaną przy użyciu kolejki priorytetowej (`std::priority_queue`). W każdym kroku rozgłaszany jest węzeł o najmniejszej wartości dolnego ograniczenia LB .
- **Cechy szczególne:** Algorytm dokładny, gwarantujący znalezienie rozwiązania optymalnego. Dzięki odcinaniu gałęzi, dla których $LB \geq UB$, przeszukuje tylko obiecujące fragmenty drzewa, co pozwala rozwiązywać większe instancje niż przegląd zupełny.

2.6 Algorytm meta-heurystyczny - Akceptacja Progu

- **Zasada działania:** Algorytm rozpoczyna pracę od permutacji początkowej, po czym w pętli generuje nowe rozwiązania z otoczenia. Zmiana (pogorszenie) wartości kryterium jest akceptowana, jeśli jest mniejsza od aktualnego progu T . Próg ten jest stopniowo obniżany w każdej iteracji o współczynnik α - geometryczny schemat chłodzenia.
- **Parametry i kryterium stopu:** Kryterium stopu stanowi maksymalna liczba iteracji zadana przez użytkownika. Początkowy próg ustalany jest jako 5% wartości początkowej, a próg minimalny wynosi 0.001.

- **Cechy szczególne:** Pozwala na ucieczkę z minimów lokalnych poprzez akceptowanie gorszych rozwiązań na początkowych etapach działania algorytmu.

2.7 Algorytm meta-heurystyczny - VNS (Variable Neighborhood Search)

- **Zasada działania:** Algorytm przeszukiwania ze zmiennym otoczeniem, który w pętli głównej na początku wykonuje tzw. fazę zaburzenia (*Shaking*) aktualnego rozwiązania w zależności od poziomu sąsiedztwa k , a następnie uruchamiane jest przeszukiwanie lokalne (*Local Search*) w celu znalezienia lokalnego optimum. Jeśli znaleziono lepsze rozwiązanie globalne, algorytm powraca do pierwszego otoczenia. W przeciwnym wypadku zwiększa stopień zaburzenia.
- **Sposoby zaburzania otoczenia (Shaking):** Zaimplementowano trzy struktury otoczenia: dla $k = 1$ zamiana dwóch losowych zadań miejscami (*Swap*), dla $k = 2$ usunięcie losowego zadania i wstawienie go na inną pozycję (*Insert*), dla $k = 3$ odwrócenie kolejności podsekwencji zadań pomiędzy dwoma losowymi indeksami (*Reverse*).
- **Kryterium stopu:** Wykonanie określonej przez użytkownika maksymalnej liczby iteracji algorytmu głównego.

3 Testy eksperymentalne

Przeprowadzono testy na algorytmach w celu analizy porównawczej ich wydajności czasowej oraz jakości generowanych rozwiązań przez algorytmy w oparciu o wyznaczone wartości kryterium C_{max} . Testy wykonano na procesorze intel core i7-5600U, oraz 8GB pamięci RAM.

3.1 Wyniki testów działania algorytmów dla $m > 2$

Przeprowadzono testy dla różnych algorytmów na różnych wielkościach instancji i różnej ilości procesów wykonujących zadania. Wyniki przedstawia tabela 1:

Tabela 1: Wyniki eksperymentów

Rozmiar instancji X ilość maszyn	Przebieg zupełny	Czas działania P.Z. [ms]	NEH	Czas działania NEH [ms]	FNEH	Czas działania FNEH [ms]	B&B	Czas działania B&B[ms]	Akceptacja Progu (linii iteracji)	Czas działania Akceptacja progui [ms]	VNS 10/100/1000 iteracji 70/70/40 % / 70/40 %	Czas działania [ms] VNS 10/100/1000 iteracji
8x5	704	59.336	705[0.142 %]	0.076	705[0.142 %]	0.053	704	14.577	704[0. %]	1657.6	707[0.426 %] / 704[0. %] / 704[0. %]	2.473 / 23.411 / 211.647
8x10	869	86.347	860[1.266 %]	0.16	860[1.266 %]	0.114	869	2.256	869[0. %]	2296.5	869[0. %] / 869[0. %] / 869[0. %]	6.014 / 42.310 / 425.410
8x20	1650	160.5	1697[2.848 %]	0.232	1697[2.848 %]	0.161	1650	49.386	1650[0. %]	3960.35	1650[0. %] / 1650[0. %] / 1650[0. %]	8.128 / 69.916 / 677.908
9x5	730	458.647	732[0.274 %]	0.141	732[0.274 %]	0.111	730	40.063	730[0. %]	1559.63	730[0. %] / 730[0. %] / 730[0. %]	4.055 / 34.707 / 316.582
9x10	925	841.131	943[1.946 %]	0.12	943[1.946 %]	0.087	925	10.163	925[0. %]	2489.4	933[0.865 %] / 925[0. %] / 925[0. %]	7.598 / 66.055 / 602.611
9x20	1695	105.85	1752[3.363 %]	0.178	1752[3.363 %]	0.116	1695	222.456	1695[0. %]	4568.82	1695[0. %] / 1695[0. %] / 1695[0. %]	9.040 / 101.911 / 1022.47
10x5	769	5061.88	777[1.040 %]	0.201	777[1.040 %]	0.123	769	94.443	769[0. %]	4.039	769[0. %] / 769[0. %] / 769[0. %]	6.328 / 55.713 / 500.96
10x10	972	8645.61	972[0. %]	0.151	972[0. %]	0.1	972	27.316	972[0. %]	2841.37	972[0. %] / 972[0. %] / 972[0. %]	14.599 / 91.687 / 966.708
10x20	1767	15604.7	1799[1.811 %]	0.23	1799[1.811 %]	0.141	1767	572.318	1767[0. %]	4761.12	1773[0.340 %] / 1773[0.340 %] / 1773[0.340 %]	15.052 / 149.839 / 1476.94
20x5	-	-	1286[0.626 %]	0.97	1286[0.626 %]	0.431	-	-	1278[0. %]	2753.83	1297[1.487 %] / 1278[0. %] / 1278[0. %]	33.954 / 358.203 / 3549.71
20x10	-	-	1502[5.849 %]	1.548	1502[5.849 %]	0.494	-	-	1426[0.493 %]	4546.34	1467[9.383 %] / 1447[9.973 %] / 1442[10.211 %]	146.88 / 960.951 / 8222.79
20x20	-	-	2397[4.627 %]	2.556	2397[4.627 %]	1.002	-	-	2307[0.698 %]	9061.95	2321[1.309 %] / 2307[0.698 %] / 2239[0.500 %]	242.538 / 1746.34 / 15492.9
50x5	-	-	2868[0.175 %]	8.245	2868[0.175 %]	1.562	-	-	2864[0.035 %]	5527.03	2872[0.314 %] / 2864[0.035 %] / 2863[0. %]	459.799 / 2967.68 / 30606.5
50x10	-	-	3159[4.915 %]	12.046	3159[4.915 %]	2.754	-	-	3017[0.199 %]	10103.6	3030[0.631 %] / 3021[0.332 %] / 3017[0.199 %]	2675.78 / 19365.4 / 136910
100x5	-	-	5261[0.210 %]	39.605	5261[0.210 %]	5.02	-	-	5253[0.057 %]	9784	5255[0.095 %] / 5253[0.095 %] / 5255[0.095 %]	2663.35 / 19363.8 / 176465

Na podstawie wyników można zaobserwować:

- Algorytm przeglądu zupełnego działa sprawnie jedynie dla najmniejszych z instancji. Już przy rozmiarze 9x5 czas obliczeń wzrasta do 458.65 ms, a dla instancji 10x20 już ponad 15 sekund. W przypadku instancji o większej liczbie zadań nie testowano algorytmu, ze względu na ograniczenia sprzętowe i olbrzymi czas obliczeń (złożoność $n!$).
- Algorytm B&B wykazuje cechy złożoności wykładniczej, lecz pozwala na szybsze obliczenie optymalnego rozwiązania niż algorytm PZ. W przypadku instancji 10x20 wyniosło to (572.3 ms), co jest prawie 30 krotnie szybsze niż PZ. Nie wykonano testów dla większych instancji z powodu ograniczeń sprzętowych i dłuższego czasu wykonania obliczeń.
- Algorytmy heurystyczne NEH i FNEH są najszybszymi algorytmami, a co istotne algorytm FNEH, który jest przyspieszoną modyfikacją NEH jest zawsze szybszy i nie pogarsza wyników (zawsze wyznacza dokładnie tę samą wartość kryterium). Dla największej instancji 100x5 NEH potrzebuje 39.6ms, a FNEH zaledwie 5.02ms. Błąd względnych tych algorytmów rzadko przekracza 5%.
- Algorytm akceptacji progu we wszystkich małych instancjach daje rozwiązanie optymalne, a dla większych instancji błąd jest bardzo niewielki (mniejszy od 1%). Wynika to przez wybraną ilość iteracji równą 1mln. w przypadku mniejszej ilości iteracji błąd może wzrosnąć, ale czas obliczeń zmaleje (trzeba dokonać kompromisu). Ponad to algorytm jest niedeterministyczny - wynik może się różnić ze względu na stosowanie funkcji random.
- W przypadku algorytmu metaheurystycznego VNS widać liniową zależność od liczby iteracji. W przypadku instancji 100x5 czas odpowiednio wzrasta od 2.6s (10 iteracji), przez 19.3s (100 iteracji) do 176.4s (1000 iteracji). Niemniej jednak zwiększenie iteracji pozwala na skuteczne wyjście z minimum lokalnych. Najlepiej można to zauważyć na instancji 20x10, gdzie błąd zmalał z 3.383% aż do 0.211%. Również jak algorytm akceptacji progu, algorytm VNS jest niedeterministyczny.

3.2 Wyniki testów dla algorytmu Johnsona dla $m=2$

Przeprowadzono testy dla algorytmu Johnsona, którego działanie jest na dwóch procesach. Wyniki przedstawia tabela 2:

Tabela 2: Wyniki eksperymentów dla Johnsona

Rozmiar instancji X 2 maszyny	Johnson	czas działania Johnson [ms]
8x2	416	0.021
9x2	426	0.02
10x2	439	0.23
20x2	930	0.036
50x2	2628	0.07
100x2	5066	0.075
200x2	10309	0.204
500x2	24920	0.257
1000x2	50057	0.527

Na podstawie wyników można zaobserwować:

- Algorytm Johnsona niemal natychmiastowo oblicza wynik dla najmniejszych instancji - czas to zaledwie 0.021 ms.
- W przypadku większych instancji czas również jest bardzo krótki - dla 1000 zadań to tylko 0.527 ms.
- Ponad to algorytm w tak krótkim czasie znajduje optymalne rozwiązanie, ponieważ taka jest jego charakterystyka, że gwarantuje znalezienie najlepszego rozwiązania.

4 Wnioski

- Badania w pełni potwierdzają teoretyczne założenia o złożoności algorytmów. Algorytmy przeglądu zupełnego i schemat B&B są bezużyteczne dla dużych instancji z powodu bariery czasowej i pamięciowej.
- Algorytmy NEH i FNEH to doskonały kompromis pomiędzy czasem a jakością wyniku. Wykonują się w bardzo krótkim czasie a ich błąd to zaledwie kilka procent. Są to idealne narzędzia jako algorytmy generujące dobra rozwiązania startowe dla metaheurystyk.
- Modyfikacja NEH do wersji FNEH przynosi oczekiwany rezultat - nie pogarsza wyniku oraz przyspiesza obliczenia.

- Algorytmy metaheurystyczne akceptacji progu i VNS pozwalają na znalezienie optymalnych rozwiązań lub ekstremalnie bliskich optymalnym (błąd poniżej 1%) dla dużych instancji.
- Wybór algorytmu w praktyce zależy od priorytetu systemu. W przypadku gdzie kluczowy jest czas reakcji należy stosować algorytm FNEH. W przypadku długoterminowości, gdy zależy nam na maksymalizacji zysku i minimalizacji kar za nieterminowość, najlepszym wyborem jest algorytm Akceptacji progu lub VNS.